TAREA SESION 5 IKER NUÑEZ UO306425

1) DyV por Sustracción

Complejidades de los siguientes programas:

Complejidad Sustraccion1 -> O(n) porque a = 1 (1 llamada recursiva), b = 1 (hace n – 1 en cada llamada) y k = 0 (porque la complejidad sin contar la llamada recursiva es O(1)). Por tanto, siguiendo la fórmula para a = 1, O(n^k+1) → O(n)

Complejidad Sustraccion2 -> en este caso, a = 1 (1 llamada recursiva), b = 1 (hace n – 1) y k = 1 (porque sin recursividad es O(n). Por tanto, siguiendo la fórmula para a = 1, O(n^k+1) → O(n^2)

Complejidad Sustraccion3 -> en este caso, a = 2 (2 llamadas recursivas), b = 1 (hace n – 1 en cada llamada) y k = 0 (porque sin recursividad es O(1)), luego siguiendo la fórmula para a > 2 → O(2^n/1) = O(2^n)

Los tiempos coinciden porque Sustraccion1 y Sustraccion2 rompen muy rápido porque cogen n muy grandes, pero porque la pila de Java tiene un tamaño limitado, entonces cuando hay tantas llamadas recursivas, rompe. Con Sustraccion3, pasa lo mismo de forma mas agresiva, ya que la complejidad es exponencial en este caso, pero como coge n's muy pequeños tarda más en cascar.

<u>¿Para qué n abortan ejecución Sustraccion1 y Sustraccion2?</u>
Sustraccion1 -> n = 8000
Sustraccion2 -> n = 8000
Ocurre por lo comentado antes, la pila de java se llena y da StackOverflow.

<u>¿Cuántos años tarda en finalizar Sustraccion3 para n = 80?</u>
La complejidad de Sustraccion3 es O(2^n). Luego,
n1 = 32, t1 = 177744ms, n2 = 80 -> t2 = (2^80/2^32) * 177744 ms = 5.0030 * 10^19 ms.
Para saber el número de años, habría que aplicar el siguiente factor de conversión:
5.0030 * 10^19 * 1 sec / 1000 ms * 1 hora / 3600 sec * 1 dia/24 horas + 365 = **2.1135 * 10^14 años**.

Sustracción 4 -> Implementación y tiempos:

```
public static void rec2 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++) cont++; // O(n)
        rec2 (n-1);
        for (int i=0;i<n;i++) cont++; // O(n)
    }
}
```

n	t(ms)
1000	12
2000	58
4000	215
8000	862
16000	fdt

Sustracción 5 -> Implementación y tiempos:

```
public static void rec2 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        rec2 (n-2);
        rec2 (n-2);
        rec2 (n-2);
    }
}
```

n	t(ms)
10	0
20	4
40	267122
80	fdt

Lo pare porque al tener una complejidad tan mala, con n's muy pequeños ya tarda mucho.

<u>¿Cuánto tardaría en finalizar la ejecución para n = 80 para Sustraccion5?</u>
Regla de 3 para n1, t1, y n2 = 80, usando la complejidad de Sustraccion5 -> O(3^n/2)

2)DyV por División

Complejidades de los siguientes programas:

Division1 -> O(n) porque a = 1, b = 3 y k = 1 -> a < b^k → O(n^1) = O(n)

Division2 -> O(n * log n) porque a = 2, b = 2 y k = 1 -> a = b^k → O(n * log n)

Division3 -> porque a = 2, b = 2 y k = 0 -> a > b ^ k → O(n^log2(2))

Division4 -> Implementación y tiempos:

```
public static void rec4 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            cont++ ; // O(1)
            for(int i = 0; i < n; i++){
                for(int j = 0; j < n; j++){

                }
            }
            rec4 (n/2);
        }

    }
```

n	t(ms)
1000	11
2000	32
4000	126
8000	504
16000	2044
32000	9885
64000	50540
120000	176178

División5 -> Implementación y tiempos:

```
public static void rec5 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            cont++ ; // O(1)
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
        }

    }
```

n	t(ms)
1000	46
2000	183
4000	833
8000	3283
16000	22769
32000	80135
64000	306691

3)Dos ejemplos básicos

Complejidades de los siguientes programas:
VectorSuma:
Sum1 -> O(n)

Sum2 -> a = 1 (hay una llamada recursiva), b = 1 (porque hace i + 1) y k = 0 (porque fuera de la recursividad es O(1)). Luego, como a = b^k, entonces O(n^k * log n), pero como k = 0 → O(log n)

Sum3 -> a = 2 (hay una llamada recursiva), b = 2 (divide entre 2) y k = 0 (fuera de la recursividad es O(1)). Luego, como a > b^k, entonces O(n^(log2 2)).

Tiempos de VectorSuma:

Suma	Sum1	Sum2	Sum3
532	680	225	4155

Fibonacci:
Fib1 -> O(n)
Fib2 -> O(n)
Fib3 -> a = 1 (1 llamada recursiva), b = 1 (porque va haciendo n – 1 en la llamada) y k = 0 (no hay mas que operaciones O(1) fuera). Por tanto, como es por sustracción -> a = 1 →O(n^k+1) = O(n)

Fib4 -> por sustracción. Hay dos llamadas recursivas -> a = 2, b no lo podríamos determinar de forma fija ya que en un caso es 1 y en otro 2, y k = 0 ya que fuera solo hay operaciones O(1). Luego, a > 1 → O(a^n/b) = O(2^n/b) donde b variable. Podríamos decir que la complejidad esta entre: O(2^n) y O(2^n/2)

Tiempos de Fibonacci:

n	Suma	Fib1	Fib2	Fib3	Fib4
40	102334155	329	472	891	5957

4)Practica DyV

Empezamos implementando el algoritmo trivial que busca la distancia mínima. Probamos con los diferentes ficheros y vemos que sale correcto.

A continuación, se presenta la implementación:

```
public class PuntosTrivial
{
    static double[][] matrizPuntos;
    static int tamañoMatriz;
    static ArrayList<String> lineas = new ArrayList<String>();
    static Random rand = new Random();

    /*
     * Metodo para leer de fichero
     * @param nombreFichero
     */
    public static void leerFichero(String nombreFichero){
        BufferedReader bf = null;
        try
        {
            bf = new BufferedReader(new FileReader(nombreFichero));
            String linea;
            boolean isFirstLine = true;
            int indice = 0;

            while((linea = bf.readLine()) != null)
            {
                if(isFirstLine)
                {
                    tamañoMatriz = Integer.parseInt(linea);
                    matrizPuntos = new double[tamañoMatriz][2];
                    isFirstLine = false;
                } else
                {
                    String[] lineaDiv = linea.split(regex: ",S");
                    matrizPuntos[indice][0] =
                        Double.parseDouble(lineaDiv[0]);
                    matrizPuntos[indice][1] =
                        Double.parseDouble(lineaDiv[1]);
                    indice++;
                }

            }
```

```java
public class PuntosTrivial
    public static void leerFichero(String nombreFichero){
        } catch (IOException e)
        {
            e.printStackTrace();
        } finally
        {
            try{
                if(bf != null)
                {
                    bf.close();
                }
            } catch(IOException e)
            {
                e.printStackTrace();
            }
        }
    }

    public static void mezclar()
    {
        for(int i = 0; i < matrizPuntos.length; i++)
        {
            int j = rand.nextInt(i + 1);

            double[] temporal = matrizPuntos[i];
            matrizPuntos[i] = matrizPuntos[j];
            matrizPuntos[j] = temporal;
        }
    }

    public static String[] BuscarDistanciaMinima()
    {
        double minDistancia = 100000000;
        int indice1 = -1;
        int indice2 = -1;
        String[] resultado = new String[2];

        for(int i = 0; i < matrizPuntos.length; i++)
        {
```

```java
public class PuntosTrivial

    public static String[] BuscarDistanciaMinima()
    {
        double minDistancia = 100000000;
        int indice1 = -1;
        int indice2 = -1;
        String[] resultado = new String[2];

        for(int i = 0; i < matrizPuntos.length; i++)
        {
            for(int j = i + 1; j < matrizPuntos.length - 1; j++)
            {
                double x1 = matrizPuntos[i][0];
                double y1 = matrizPuntos[i][1];

                double x2 = matrizPuntos[j][0];
                double y2 = matrizPuntos[j][1];

                //formula de la distancia entre 2 puntos: Raiz((x2-x1)^2 + (y2-y1)^2)
                double distancia = Math.sqrt((Math.pow((x2 - x1), b: 2)) + Math.pow((y2 - y1), b: 2));

                if(distancia < minDistancia){
                    minDistancia = distancia;
                    indice1 = i;
                    indice2 = j;
                }
            }
        }

        resultado[0] = String.format(format: "SU DISTANCIA MINIMA: %.6f", minDistancia);
        resultado[1] = String.format(format: "PUNTOS MAS CERCANOS: [%.6f,%.6f] [%.6f,%.6f]",
            matrizPuntos[indice1][0], matrizPuntos[indice1][1],
            matrizPuntos[indice2][0], matrizPuntos[indice2][1]);

        return resultado;
    }

}
```

La complejidad es claramente O(n^2) ya que tenemos dos bucles for anidados.

Y una clase PuntosTrivialTiempos que repite el algoritmo i veces y mide los tiempos de ejecución:

```java
import p3p.PuntosTrivial;

public class PuntosTrivialTiempos
{
    Run | Debug
    public static void main(String[] args)
    {
        String nombreFichero = args[0];

        long t1,t2;
        String[] resultado;
        PuntosTrivial.leerFichero(nombreFichero);

        for (int i = 1024; i <= 1000000; i *= 2)
        {
            //Ordenamos las filas de forma aleatoria
            PuntosTrivial.mezclar();

            t1 = System.currentTimeMillis ();

            resultado = PuntosTrivial.BuscarDistanciaMinima();
            System.out.println(resultado[0]);
            System.out.println(resultado[1]);

            t2 = System.currentTimeMillis ();

            System.out.println (" i="+i+ "**TIEMPO="+(t2-t1));

        }
    }
}
```

Con este programa, obtuve los siguientes resultados para los diferentes ficheros:

Datos32:
SU DISTANCIA MINIMA: 0,931098
PUNTOS MAS CERCANOS: [27,278446,90,131051] [27,536642,89,236468]

Tiempos:

n	Tiempos
1024	31
2048	0
4096	1
8192	0
16384	1
32768	1
65536	0
131072	2
262144	0
542288	1

Datos128:
SU DISTANCIA MINIMA: 0,726086
PUNTOS MAS CERCANOS: [91,160139,17,465655] [91,434401,16,793360]
Tiempos:

n	Tiempos
1024	35
2048	1
4096	1
8192	1
16384	2
32768	2
65536	1
131072	1
262144	1
542288	1

Datos512:
SU DISTANCIA MINIMA: 0,039745
PUNTOS MAS CERCANOS: [68,511846,94,653169] [68,474715,94,638993]
Tiempos:

n	Tiempos
1024	42
2048	2
4096	4
8192	1
16384	4
32768	2
65536	2
131072	2
262144	2
542288	1

Datos2048:
SU DISTANCIA MINIMA: 0,036184
PUNTOS MAS CERCANOS: [27,692774,55,783120] [27,686908,55,818825]
Tiempos:

n	Tiempos
1024	60
2048	20
4096	7
8192	7
16384	6
32768	7
65536	7
131072	7
262144	8
542288	6

Datos8192:
SU DISTANCIA MINIMA: 0,009964
PUNTOS MAS CERCANOS: [83,681527,10,289108] [83,691485,10,288760]
Tiempos:

n	Tiempos
1024	162
2048	116
4096	97
8192	97
16384	114
32768	121
65536	105
131072	123
262144	110
542288	109

Ahora, los tiempos para una matriz generada de forma aleatoria:

n	t(ms)
1024	45
2048	35
4096	25
8192	96
16384	429
32768 fdt	

Ahora, el algoritmo de Divide y Vencerás:
A continuación, se muestra la implementación:

```java
public static String[]  BuscarDistanciaMinima()
{
    int iz = 0;
    int de = matrizPuntos.length - 1;

    quickSortPorX();

    Object[] resultado = BuscarDistanciaMinimaRec(iz, de);
    double distanciaMinima = (Double) resultado[0];
    int i = (Integer) resultado[1];
    int j = (Integer) resultado[2];

    String[] retorno = new String[2];

    retorno[0] = String.format(format: "SU DISTANCIA MINIMA: %.6f", resultado[0]);
    retorno[1] = String.format(format: "PUNTOS MAS CERCANOS: [%.6f,%.6f] [%.6f,%.6f]",
        matrizPuntos[i][0], matrizPuntos[i][1],
        matrizPuntos[j][0], matrizPuntos[j][1]);

    return retorno;

}
```

```java
public static Object[] BuscarDistanciaMinimaRec(int iz, int de)
{
    double minDistancia = Double.MAX_VALUE;
    int indice1 = -1;
    int indice2 = -1;
    Object[] resultado = new Object[3];

    Object[] resultado1;
    Object[] resultado2;

    Object[] retorno;

    int indiceParticion = (iz + de)/2;

    if(de == iz)
    {
        resultado[0] = minDistancia;
        resultado[1] = indice1;
        resultado[2] = indice2;
        return resultado;
    }

    if(de - iz == 1)
    {
        double x1 = matrizPuntos[iz][0];
        double y1 = matrizPuntos[iz][1];

        double x2 = matrizPuntos[de][0];
        double y2 = matrizPuntos[de][1];

        //formula de la distancia entre 2 puntos: Raiz((x2-x1)^2 + (y2-y1)^2)
        double distancia = Math.sqrt((Math.pow((x2 - x1), b: 2)) + Math.pow((y2 - y1), b: 2));

        if(distancia < minDistancia)
        {
            minDistancia = distancia;
            indice1 = iz;
```

```
        if(distancia < minDistancia)
        {
            minDistancia = distancia;
            indice1 = iz;
            indice2 = de;

            resultado[0] = minDistancia;
            resultado[1] = indice1;
            resultado[2] = indice2;

            return resultado;
        }
    }

    resultado1 = BuscarDistanciaMinimaRec(iz, indiceParticion);

    resultado2 = BuscarDistanciaMinimaRec(indiceParticion + 1, de);

    retorno = combinar(resultado1, resultado2);

    return retorno;
}

public static Object[] combinar(Object[] resultado1, Object[] resultado2)
{
    double distancia1 = (Double) resultado1[0];
    double distancia2 = (Double) resultado2[0];

    return distancia1 < distancia2? resultado1 : resultado2;
}
```

Y después la clase que mide los tiempos:

```
Run | Debug
public static void main(String[] args)
{
    long t1,t2;
    String[] resultado;

    for (int i = 1024; i <= 10000000; i *= 2)
    {
        PuntosDyV.generarMatrizPuntosAleatoria(i);

        t1 = System.currentTimeMillis ();

        resultado = PuntosDyV.BuscarDistanciaMinima();
        System.out.println(resultado[0]);
        System.out.println(resultado[1]);

        t2 = System.currentTimeMillis ();

        System.out.println (" i="+i+ "**TIEMPO="+(t2-t1));

    }
}
```

La complejidad del algoritmo es:
a = 2
b = 2
k = 1 (O(n * log n) por Quicksort)
➔ La complejidad es O(n * log n)

En el caso de este algoritmo, los resultados para los ficheros dados no son siempre correctos, ya que si los puntos que forman la distancia mínima caen en particiones distintas, nunca se calcula esa distancia y

entonces se devuelve una distancia que no es la mínima real. Pero, no he encontrado una solución que no se cargue la complejidad (usa fors) y la que encontré no la entiendo, luego prefiero dejarlo así.

Por esta razón, muestro los tiempos de ejecución para matrices aleatorias directamente, y no los resultados para los ficheros, ya que los resultados ya se que serán incorrectos.

Y ahora los tiempos de ejecución para matrices aleatorias creadas con los siguientes tamaños n:

n	t(ms)
1024	41
2048	2
4096	3
8192	4
16384	8
32768	19
65536	35
131072	107
262144	189
524288	578
1048576	fdt

Diría que los tiempos de cada algoritmo se corresponden con la complejidad de cada algoritmo.